

Assignment 9: RFT

Introduction

This project implements a Reliable File Transfer (RFT) protocol built on top of raw UDP sockets in Python. The goal is to provide reliable end-to-end file transfer over an inherently unreliable transport layer, without relying on the operating system's TCP/IP stack for reliability guarantees.

The implementation consists of two programs: RFT_UDPServer, which serves files upon request, and RFT_UDPClient, which requests and downloads files by name. Both programs use SOCK_RAW sockets with IP_HDRINCL, meaning they manually construct IP and UDP headers for every packet using Python's struct module.

Reliability is achieved through four mechanisms: a checksum for detecting corrupted packets, sequence numbers for detecting duplicates and reordering packets, cumulative acknowledgement numbers to reduce ACK overhead, and retransmission on timeout to recover from packet loss.

The server uses a sliding window protocol with multithreading — a sender thread manages the window and transmits chunks, while a separate ACK receiver thread listens for acknowledgements and advances the window. This allows the server to keep packets in flight while waiting for acknowledgements, improving throughput.

How to Run

Prerequisites

- Python 3.10 or higher
- Both machines must be on the same network or reachable via IP

Setup

Clone the repository on both machines:

```
git clone https://github.com/dhruvdingari8/reliable-file-transfer.git
cd cs5700-rft
```

Place the file to be transferred in the same directory as RFT_UDPServer.py on the server machine.

Running the Server

```
sudo python3 RFT_UDPServer.py <server_ip>
```

Example:

```
sudo python3 RFT_UDPServer.py 192.168.1.199
```

Running the Client

```
sudo python3 RFT_UDPClient.py <client_ip> <server_ip> <filename>
```

Example:

```
sudo python3 RFT_UDPClient.py 192.168.1.168 192.168.1.199 test_100mb.bin
```

Simulating Packet Loss

To apply packet loss on the server machine before running a test:

```
sudo tc qdisc add dev <interface> root netem loss <percentage>%
```

Example for 2% loss:

```
sudo tc qdisc add dev eno1 root netem loss 2%
```

To remove packet loss after testing:

```
sudo tc qdisc del dev eno1 root
```

Design and Architecture

Packet Structure

Every packet consists of four parts:

IP Header	20 Bytes
UDP Header	8 Bytes
App Header	14 Bytes
Payload	≤1400 Bytes

The IP and UDP Headers are manually constructed using Python's struct module, specifically with big endian byte ordering. The IP header includes a checksum computed using the ones' complement algorithm. The UDP checksum is left as zero since reliability is handled at the application layer.

The application header includes the following five fields:

Sequence Number	4 Bytes	Identifies packet order
ACK Number	4 Bytes	Cumulative acknowledgement
Flags	1 Byte	Packet type (DATA/ACK/REQUEST/FIN)
Padding	1 byte	Alignment (Not used for any calculations)
Data Length	2 Bytes	Payload size in bytes

Checksum	2 Bytes	Integrity Verification
----------	---------	------------------------

Reliability Mechanisms

1. **Checksum:** computed over the application header and payload using ones' complement addition. The receiver verifies integrity by checking that the sum of all 16-bit words including the checksum field equals 0xFFFF.
2. **Sequence numbers:** each chunk is assigned a monotonically increasing sequence number starting from 0. The receiver uses these to detect duplicates and reorder out-of-order packets.
3. **Cumulative ACKs:** rather than acknowledging every packet individually, the client sends a single ACK containing the next expected sequence number. This reduces ACK traffic significantly for large transfers.
4. **Retransmission on timeout:** the server maintains a dictionary of unacknowledged packets with their send timestamps. After a fixed timeout period, any unacknowledged packet is retransmitted.

Sliding Window

The server uses a sliding window protocol to keep multiple packets in flight simultaneously. Two variables track the window state: `base` (the lowest unacknowledged sequence number) and `next_seq` (the next sequence number to send). The window allows up to `WINDOW_SIZE` packets in flight at once.

Threading Model

The server spawns two threads per file transfer:

1. **Sender thread:** fills the window with new packets, waits when the window is full, and retransmits timed-out packets after each wait
2. **ACK receiver thread:** listens for incoming ACK packets, advances `base`, and signals the sender thread via a `threading.Event`

Shared state between threads (`base`, `next_seq`, `unacked`) is protected by a `threading.Lock`. A known minor race condition exists when reading `base` and `next_seq` outside the lock for the window size check. However, in the worst case this causes an unnecessary wait but does not affect correctness. Hence, it was deemed fine to ignore.

Key Functionalities

Checksum

The checksum is computed using the ones' complement algorithm over the application header and payload. The sender computes the checksum with the checksum field zeroed out, then embeds the result in the header before sending. The receiver recomputes the checksum over the entire application header including the embedded checksum field. A correct packet will produce 0xFFFF. Packets that fail this check are silently discarded, triggering a retransmission via timeout.

A special case is handled where a checksum of 0x0000 is replaced with 0xFFFF per the ones' complement convention.

Sequence Numbers

Each chunk of file data is assigned a monotonically increasing sequence number starting at 0. The receiver stores chunks in a dictionary keyed by sequence number, allowing out-of-order packets to be buffered and correctly reassembled later. Duplicate packets are naturally handled, such that if a sequence number already exists in the dictionary, the new chunk simply overwrites the old one with identical data.

Cumulative Acknowledgements

Rather than sending one ACK per packet, the client sends a cumulative ACK after each received chunk. The ACK number represents the next sequence number the client expects, meaning all chunks below that number have been received. This allows a single ACK to acknowledge multiple packets at once when chunks arrive in order, significantly reducing ACK traffic for large transfers.

Retransmission on Timeout

The server maintains an unacked dictionary, mapping sequence numbers to (packet, timestamp) tuples. After waiting for an ACK, the server checks all entries in unacked. Any packet whose timestamp exceeds the fixed timeout value is retransmitted and its timestamp is reset. The timeout value is fixed at 0.5 seconds.

File Integrity Verification

Both the server and client compute an MD5 hash of their respective files. The server hashes the original file before sending, and the client hashes the reassembled file after receiving. These hashes are printed in the Client's

Report Generation

After each transfer the server automatically generates a text report in the output/ directory containing the filename, file size, packet loss percentage, packets sent, packets retransmitted, packets received, transfer duration, and MD5 hashes of both the original and received files.

Testing

Test Environment

Testing was conducted between two Linux machines on a local area network:

Server: HP Z420 Workstation running Ubuntu 24.04, IP 192.168.1.199, connected via Ethernet

Client: Dell Precision 5570 running EndeavourOS, IP 192.168.1.168, connected via WiFi

Packet loss was simulated on the server machine using the tc netem traffic control utility:
sudo tc qdisc add dev eno1 root netem loss <percentage>%

Note: Testing was performed on a local network rather than AWS EC2 due to AWS account quota restrictions that prevented launching EC2 instances. The functionality is identical — the same tc netem commands specified in the assignment appendix were used to simulate packet loss.

Test Files

Three test files of random binary data were generated using dd:

dd if=/dev/urandom of=test_100mb.bin bs=1M count=100

dd if=/dev/urandom of=test_500mb.bin bs=1M count=500

dd if=/dev/urandom of=test_1gb.bin bs=1M count=1024

Results

0% Packet Loss

File	Size	Packets Sent	Retransmitted	Duration
test_100mb.bin	104,857.6KB	74899	832	00:00:49
test_500mb.bin	524,288.0KB	374492	2048	00:03:25

test_1gb.bin	1,073,741.824KB	766959	9984	00:08:44
--------------	-----------------	--------	------	----------

2% Packet Loss

File	Size	Packets Sent	Retransmitted	Duration
test_100mb.bin	104,857.6KB	74899	237585	00:10:02
test_500mb.bin	524,288.0KB	374492	1277519	00:52:42
test_1gb.bin	1,073,741.824KB	1141451	3924401	01:49:13

4% Packet Loss

File	Size	Packets Sent	Retransmitted	Duration
test_100mb.bin	104,857.6KB	74899	319807	00:12:57
test_500mb.bin	524,288.0KB	374492	1551510	01:03:52
test_1gb.bin	1,073,741.824KB	1141451	4248768	02:14:38

All transfers completed with matching MD5 hashes, confirming correct file reassembly under all test conditions.

Screenshots

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_100mb_0_pacloss_report.txt
RFT Transmission Report
-----
File Info
-----
Filename:      test_100mb.bin
Size:         104857600
-----
Packet Stats
-----
Packet Loss %:    0%
Packets Sent:     74899
Packets Retransmitted: 832
Packets Received: 76696
Total Time Taken: 00:00:49
-----
Hashes
-----
MD5 Hash of Original File: 9ef9a727f7916c46c93d770323dd613b
MD5 Hash of Received File: 9ef9a727f7916c46c93d770323dd613b
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_500mb_0_pacloss_report.txt
RFT Transmission Report
-----
File Info
-----
Filename:      test_500mb.bin
Size:         524288000
-----
Packet Stats
-----
Packet Loss %:    0%
Packets Sent:     374492
Packets Retransmitted: 2048
Packets Received: 375719
Total Time Taken: 00:03:25
-----
Hashes
-----
MD5 Hash of Original File: 1327abee81e931d02ce92fa34778bd7
MD5 Hash of Received File: 1327abee81e931d02ce92fa34778bd7
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_1gb_0_pacloss_report.txt
RFT Transmission Report
-----
                        File Info
-----
Filename:                test_1gb.bin
Size:                    1073741824
-----
                        Packet Stats
-----
Packet Loss %:           0%
Packets Sent:            766959
Packets Retransmitted:   9984
Packets Received:       772300
Total Time Taken:        00:08:44
-----
                        Hashes
-----
MD5 Hash of Original File:  f0f8e90ed4f2a750a1aaee858424d90e
MD5 Hash of Received File:  f0f8e90ed4f2a750a1aaee858424d90e
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_100mb_2_pacloss_report.txt
RFT Transmission Report
-----
                        File Info
-----
Filename:                test_100mb.bin
Size:                    104857600
-----
                        Packet Stats
-----
Packet Loss %:           2%
Packets Sent:            74899
Packets Retransmitted:   237585
Packets Received:       121146
Total Time Taken:        00:10:02
-----
                        Hashes
-----
MD5 Hash of Original File:  9ef9a727f7916c46c93d770323dd613b
MD5 Hash of Received File:  9ef9a727f7916c46c93d770323dd613b
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```



```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_500mb_2_pacloss_report.txt
RFT Transmission Report
-----
                        File Info
-----
Filename:                test_500mb.bin
Size:                    524288000
-----
                        Packet Stats
-----
Packet Loss %:           2%
Packets Sent:            374492
Packets Retransmitted:   1277519
Packets Received:        611772
Total Time Taken:        00:52:42
-----
                        Hashes
-----
MD5 Hash of Original File: 1327abee81e931d02ce92fa34778bd7
MD5 Hash of Received File: 1327abee81e931d02ce92fa34778bd7
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_1gb_2_pacloss_report.txt
RFT Transmission Report
-----
                        File Info
-----
Filename:                test_1gb.bin
Size:                    1073741824
-----
                        Packet Stats
-----
Packet Loss %:           2%
Packets Sent:            1141451
Packets Retransmitted:   3924401
Packets Received:        1867559
Total Time Taken:        01:49:13
-----
                        Hashes
-----
MD5 Hash of Original File: f0f8e90ed4f2a750a1aaee858424d90e
MD5 Hash of Received File: f0f8e90ed4f2a750a1aaee858424d90e
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_100mb_4_pacloss_report.txt
RFT Transmission Report
-----
File Info
-----
Filename:      test_100mb.bin
Size:         104857600
-----
Packet Stats
-----
Packet Loss %:      4%
Packets Sent:       74899
Packets Retransmitted: 319807
Packets Received:   134503
Total Time Taken:   00:12:57
-----
Hashes
-----
MD5 Hash of Original File: 9ef9a727f7916c46c93d770323dd613b
MD5 Hash of Received File: 9ef9a727f7916c46c93d770323dd613b
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

```
chaewon@blaze-z420:~/src/CS5700/rft$ cat output/test_500mb_4_pacloss_report.txt
RFT Transmission Report
-----
File Info
-----
Filename:      test_500mb.bin
Size:         524288000
-----
Packet Stats
-----
Packet Loss %:      4%
Packets Sent:       374492
Packets Retransmitted: 1551510
Packets Received:   665937
Total Time Taken:   01:03:52
-----
Hashes
-----
MD5 Hash of Original File: 1327abee81e931d02ce92fa34778bd7
MD5 Hash of Received File: 1327abee81e931d02ce92fa34778bd7
-----
chaewon@blaze-z420:~/src/CS5700/rft$
```

Limitations

Cascading Retransmissions

The most significant performance limitation is cascading retransmissions under packet loss. With a fixed timeout and no exponential backoff, a single lost packet can trigger repeated retransmissions of the same packet and all subsequent unacknowledged packets in the window. This results in a disproportionately high retransmission count. For example, a 1GB transfer with 2% packet loss produced nearly 4 million retransmissions and took nearly 2 hours compared to under 9 minutes with no loss. A real TCP implementation would use exponential backoff to avoid flooding the network during congestion.

Fixed Timeout

The timeout value is fixed at 0.5 seconds regardless of network conditions. On a low-latency LAN this causes unnecessary retransmissions since packets are typically delivered in milliseconds. On a high-latency WAN this might cause premature retransmissions. The weighted average RTT formula used by TCP would significantly improve performance.

Loss Percentage Hardcoded

The packet loss percentage in the report is hardcoded in `handle_request` rather than passed as a command line argument. This means the server must be manually edited between test runs to reflect the correct loss percentage.

Known Race Condition

A minor race condition exists in the sender thread where `base` and `next_seq` are read outside the lock to check the window size. In the worst case this causes an unnecessary wait or a skipped wait but does not affect correctness or file integrity.

FIN Reliability

The FIN packet is sent multiple times to reduce the chance of loss, but there is no formal acknowledgement of the FIN from the client. If all FIN copies are lost, the client will hang indefinitely waiting for a FIN that never arrives.

Testing Environment

Due to AWS account quota restrictions that prevented launching EC2 instances, testing was performed on two local Linux machines connected via a home LAN rather than AWS EC2. Packet loss was simulated using `tc netem` as specified in the assignment. The code is expected to function identically on AWS EC2.

Lessons Learned

Raw Sockets Are More Complex Than Expected

Working with `SOCK_RAW` sockets revealed how much complexity the OS networking stack normally hides. Manually constructing IP and UDP headers, computing checksums, and handling byte ordering with `struct.pack` gave a much deeper understanding of how packets are actually structured on the wire. A particularly tricky issue was discovering that `SOCK_RAW` with `IPPROTO_RAW` is send-only on Linux, and receiving requires `IPPROTO_UDP` instead.

Checksum Implementation Subtleties

Implementing the ones' complement checksum correctly took significant debugging. Key lessons included the importance of even-length padding, proper overflow folding, and a special case where a computed checksum of `0x0000` must be replaced with `0xFFFF`. Additionally, the app header struct format `">IIBHH"` was initially 13 bytes (odd), causing checksum failures. Adding an explicit padding byte with `">IIBxHH"` resolved this.

Threading and Shared State

Designing the sliding window with two threads required careful thought about shared state and synchronization. Using `threading.Lock` for shared variables and `threading.Event` for signaling between threads worked well. A key lesson was avoiding holding the lock during blocking operations like `socket.settimeout` and `recvfrom`, which can cause deadlocks.

Protocol Design Decisions Matter

Small design decisions had significant performance implications. Moving the retransmit check outside the main send loop caused cascading retransmissions under packet loss. The choice of fixed timeout versus adaptive timeout dramatically affects performance under loss. These tradeoffs are why real transport protocols like TCP are so carefully engineered.

AI Tools as a Debugging Aid

Claude was used throughout this project as a debugging and learning aid, particularly for understanding raw socket behavior on Linux, diagnosing the checksum bug, and understanding the threading model. Per the assignment guidelines, all the code was hand written by the student, while Claude was used to explain concepts, identify bugs, and suggest fixes which were then understood and implemented manually.

Future Improvements

Exponential Backoff

The most impactful improvement would be implementing exponential backoff for retransmissions, doubling the timeout after each failed retransmit attempt and resetting it when an ACK is received. This would prevent the cascading retransmission problem observed under packet loss and bring performance much closer to TCP's behavior.

Adaptive Timeout

Rather than a fixed timeout, implementing the weighted average RTT formula used by TCP would allow the timeout to dynamically adjust to network conditions. This would reduce unnecessary retransmissions on low-latency networks and avoid premature retransmissions on high-latency ones.

FIN Handshake

A proper FIN/ACK handshake similar to TCP's connection teardown would eliminate the current limitation where a lost FIN causes the client to hang indefinitely. The client would send a FIN-ACK in response to the server's FIN, and the server would wait for confirmation before closing.

Selective Repeat

The current implementation retransmits all unacknowledged packets in the window when a timeout fires, similar to Go-Back-N. Implementing Selective Repeat, where only the specific lost packet is retransmitted, would significantly reduce unnecessary retransmissions and improve throughput under packet loss.

Dynamic Window Sizing

Implementing a congestion window that grows and shrinks based on network conditions (similar to TCP's slow start and congestion avoidance) would improve throughput on varying network conditions rather than using a fixed window size.

Command Line Loss Percentage

The packet loss percentage in the report is currently hardcoded. Adding it as a command line argument would make the server more flexible and eliminate the need to edit the source code between test runs.

Error Recovery

Currently if the server's file is not found or the client crashes mid-transfer, the other side hangs indefinitely. Adding proper error packets and timeouts for the overall transfer session would make the protocol more robust.